Pregunta 5: El *Kitty Problem* y el Principio de Abierto/Cerrado

Miguel Cazorla Zamora

1. Descripción del problema

El *Kitty Problem* parte de un sistema agregador de taxis dividido en servicios: `TaxiUI` (atiende al cliente), `TaxiFinder` (busca taxis disponibles), `TaxiSelector` (elige el más adecuado según coste, tiempo o lujo), `TaxiDispatcher` (ordena el servicio) y `TaxiSupervisor` (controla el viaje) (Martin, 2017). Los servicios se despliegan por separado y parecen desacoplados.

Un día se pide un requisito nuevo: la entrega de gatitos a domicilio. Hay conductores alérgicos a los gatos que deben excluirse, vehículos no aptos e instrucciones de recogida y devolución propias. El resultado es que *todos* los servicios deben modificarse, y esos cambios deben coordinarse y desplegarse juntos. Los servicios estaban desacoplados en ejecución, pero no frente al cambio (Martin, 2017).

2. ¿Por qué la división por verbos provocó el problema?

Los servicios `TaxiFinder` o `TaxiSelector` representan *pasos del algoritmo* de reserva, no conceptos de negocio. Es una descomposición funcional: el concepto "tipo de viaje" queda repartido entre todos los servicios, de modo que cuando ese concepto cambia (aparecen los gatitos), el cambio se propaga a todos ellos. Richards y Ford (2020) lo describen como una partición técnica frente a una partición por dominio: en la primera, un cambio de dominio atraviesa todas las particiones; en la segunda, queda confinado a una sola. Además, todos los servicios comparten la misma representación del viaje, por lo que siguen acoplados a través de los datos.

3. Diseño basado en DDD y Onion/Clean

DDD propone dividir el sistema según los conceptos del dominio y sus ejes de cambio. Aquí la observación clave es que la entrega de gatitos *es un tipo de viaje*: comparte el flujo general (buscar, seleccionar, despachar, supervisar) pero varía en reglas concretas. Por tanto se separan:

- **`Rides`**: núcleo con la lógica estable del viaje y los puntos de extensión (clases abstractas e interfaces como `Ride` o `DriverEligibility`).
- **`Kittens`**: plugin que implementa esas abstracciones con las reglas propias de los gatitos (p. ej. excluir conductores alérgicos).

Siguiendo la Regla de Dependencia (Martin, 2017) y la Arquitectura Onion (Palermo, 2008), `Rides` ocupa el centro y `Kittens` depende de él, nunca al revés (Figura 1). Cada servicio puede mantener su despliegue, pero internamente opera sobre las abstracciones de `Rides` mediante polimorfismo (*Template Method* o *Strategy*), sin conocer el tipo concreto de viaje (Martin, 2017). Así, añadir gatitos solo afecta a un componente: `Kittens`.

4. Principio de Abierto/Cerrado y papel de los Adaptadores

El OCP establece que un artefacto debe estar abierto a la extensión y cerrado a la modificación (Martin, 2017). En el diseño original, añadir gatitos exigía modificar cinco servicios. En el diseño

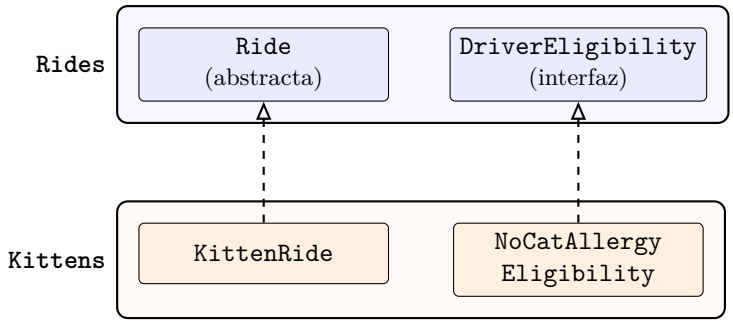


Figura 1: El plugin `Kittens` implementa las abstracciones del núcleo `Rides`. Las dependencias apuntan hacia el núcleo.

propuesto, el núcleo `Rides` y los servicios existentes no se tocan (cerrados a la modificación) y el nuevo comportamiento se añade creando el componente `Kittens` (abierto a la extensión).

Los Adaptadores hacen posible esta extensión. En términos de la Arquitectura Hexagonal (Cockburn, 2005), las abstracciones de `Rides` son *puertos* y `Kittens` es un *adaptador* que traduce las reglas de los gatitos al contrato que entiende el núcleo, que nunca conoce el concepto "gatito". Si los gatitos provinieran de un sistema externo, un adaptador de servicio encapsularía esa integración tras un puerto del dominio (De la Torre et al., 2010). Finalmente, el componente *Main* conecta el plugin mediante factorías o inyección de dependencias (Martin, 2017), por lo que incorporar un nuevo tipo de viaje se reduce a registrar un nuevo adaptador.

Referencias

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters).* `https://alistair.cockburn.us/hexagonal-architecture/`

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0* (Borrador). Microsoft Ibérica.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.

Palermo, J. (2008). *The Onion Architecture.* `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/`

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach.* O'Reilly Media.